Notice that the acceptance or rejection of a transition to a new state is not related to the distance of the new state from current one *in the problem space*: it's only based on the *energy levels* of the two states, which in turn are given by (or at least proportional to) the value of the cost function in those states.

What's even more important, after the very initial stages where basically all transitions are allowed, then the furthest uphill a new state is, the less likely it becomes for the transition to be accepted: this is why the algorithm works well, because it progressively encourages transitions towards areas where cost is lower (see figure 17.6.B-C) while not limiting the search to the neighborhood of current position in problem space.

Why do transitions uphill work? Because going uphill it might be able to go over a cliff and reach a deeper valley. While it might seem counter-intuitive on a 2D chart, this becomes even more relevant in high-dimensional space.

Similarly to the metallurgic process, however, cooling down the system with the right pace is instrumental to the quality of the final result.

And, of course, being a merely-stochastic process, it also needs some "luck": especially towards the end, many random steps will produce transitions uphill and as such be rejected. But if we try hard enough, for many iterations, chances are a positive change will be randomly found.

And that's the strength of the algorithm, and its weakness at the same time: we can reach an improvement, but since we